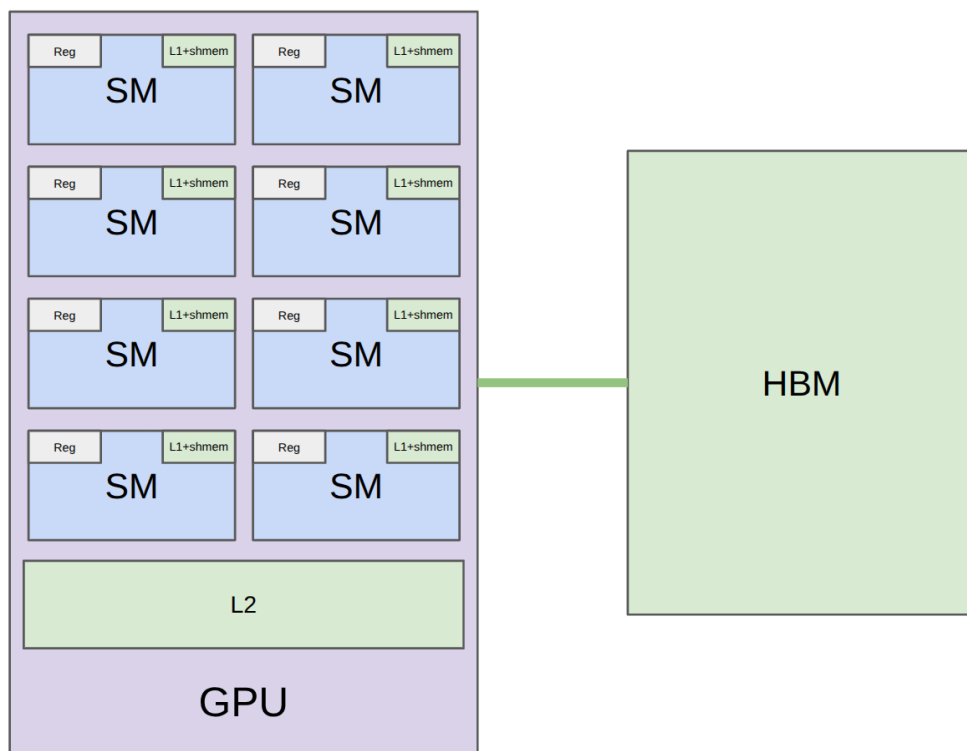


课程笔记

CS336 2026 Lecture 6: Kernels、Profiling 与 Triton

基于 Stanford CS336 Spring 2026 官方讲义整理

2026 年春季



课程/频道: Stanford CS336

发布日期: Spring 2026

材料类型: 官方可执行讲义 lecture06-slides.py

课程主页: <https://cs336.stanford.edu/>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 本讲主线：从性能猜测到 kernel 证据	2
1.1 为什么 kernel 是 LLM 系统的基本单位	3
1.2 本讲四个案例	3
1.3 本章小结	3
2 GPU 编程模型复习：正确性抽象与性能现实	4
2.1 从 grid 到 thread block	4
2.2 Warp: 32 个线程同步前进	5
2.3 Occupancy: 更多 warp 不一定等于更快	5
2.4 Shared memory bank conflict	6
2.5 HBM coalescing 与 block occupancy	6
2.6 本章小结	7
3 Benchmarking: GPU 计时首先要可信	7
3.1 为什么 CPU wall-clock 容易误导	7
3.2 Warmup 的真实含义	8
3.3 维度扫描比单点计时更有价值	8
3.4 常见计时陷阱	9
3.5 本章小结	9
4 Profiling: 从 PyTorch 表达式看到 CUDA kernel	10
4.1 Profiler 解决 benchmark 解决不了的问题	10
4.2 Kernel name 是性能线索	10
4.3 add、matmul 和 shape 依赖	11
4.4 Profiler 与 Nsight 的关系	11
4.5 本章小结	11
5 GeLU: 逐元素函数里的 fusion 账本	11
5.1 GeLU 公式与 naive PyTorch	11
5.2 内存 traffic 为什么会压过 FLOPs	12
5.3 builtin 与 torch.compile	12
5.4 本章小结	13
6 Triton 编程模型：在 thread block 层写 GPU 程序	13
6.1 CUDA 与 Triton 的抽象边界	13
6.2 一个 Triton kernel 的骨架	14
6.3 Host wrapper 与 grid	15
6.4 从 Triton 到 PTX	15
6.5 本章小结	15
7 Triton GeLU: 最小但完整的融合 kernel	16
7.1 Launch 逻辑	16
7.2 Kernel 逻辑	16

7.3	Mask 的语义	17
7.4	本章小结	17
8	Fused Softmax: 当一整行放进一个 block	17
8.1	Softmax 的稳定形式	17
8.2	Naive softmax 的 HBM 账本	17
8.3	Triton fused softmax	18
8.4	为什么越界值用负无穷	19
8.5	行能否放进 block 是关键假设	19
8.6	本章小结	20
9	Row Sum: 当一行放不进一个 block	20
9.1	从 softmax 简化到 row sum	20
9.2	Triton row sum kernel	21
9.3	从 row sum 到 online softmax	22
9.4	本章小结	22
10	Matmul + ReLU: tiling、复用与融合	22
10.1	Naive matmul 的 arithmetic intensity	22
10.2	Tiling 的基本思路	23
10.3	Triton matmul + ReLU 的 launch	24
10.4	Kernel 核心: 指针矩阵与 K 循环	24
10.5	为什么 ReLU fusion 几乎免费	26
10.6	真实 GEMM kernel 更复杂在哪里	26
10.7	本章小结	26
11	从硬件事实到工程判断	26
11.1	什么时候该写 kernel	26
11.2	性能 checklist	27
11.3	Shape 本身是系统设计参数	27
11.4	正确性测试不能只看一次 allclose	28
11.5	本章小结	28
12	拓展阅读	28
13	总结与延伸	28

1 本讲主线：从性能猜测到 kernel 证据

Lecture 5 建立了 GPU 的硬件心智模型：SM、warp、HBM、shared memory、roofline、fusion、tiling 和 FlashAttention。Lecture 6 则把这些概念落到工程动作上：怎样准确计时，怎样看到 PyTorch 实际调用了哪些 CUDA kernel，怎样判断瓶颈是 kernel launch、HBM traffic、occupancy、memory coalescing 还是 tile shape，最后怎样用 Triton 写出可控但不至于过早下沉到 CUDA C++ 的 GPU kernel。

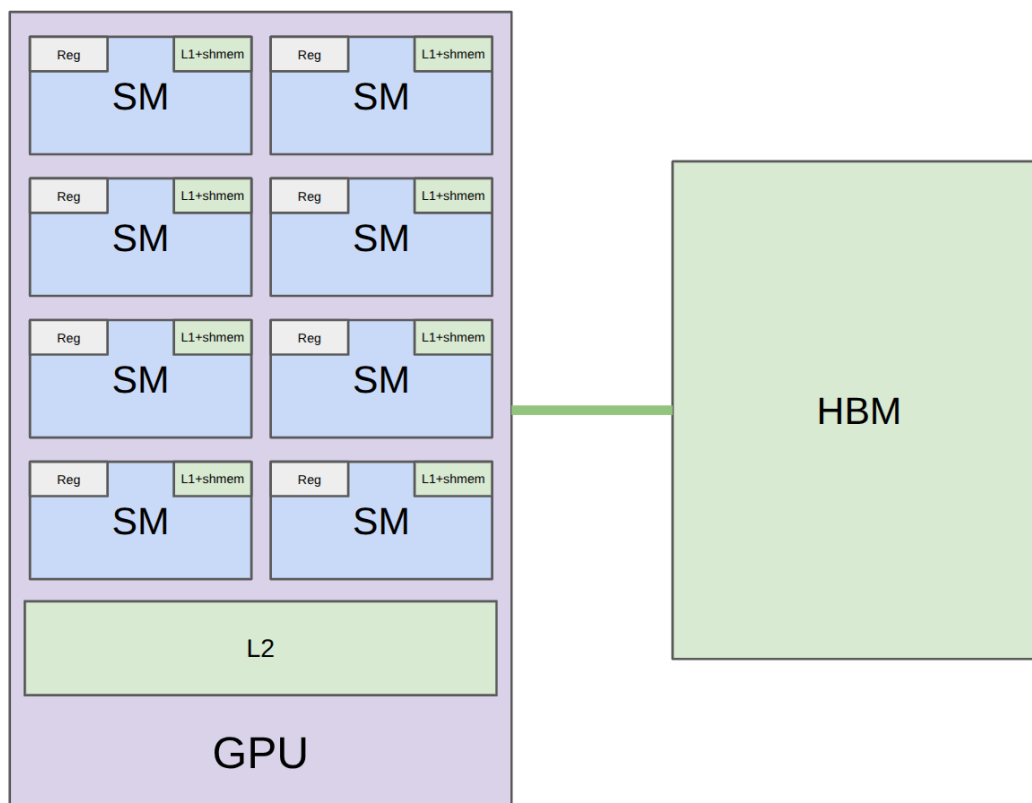


图 1: 本讲继续围绕 GPU 层次结构展开：寄存器、shared memory/L1、L2、HBM 的带宽差异决定了 kernel 优化的方向。

来源：CS336 Spring 2026 Lecture 6 官方可执行讲义引用图。

这讲最值得保留的不是某一个 Triton API，而是一套可重复的性能工程循环：

1. **Benchmark**: 先用可信计时回答“这个实现到底多快”。
2. **Profile**: 再用 profiler 回答“时间花在哪些 kernel 上”。
3. **Read the kernel**: 观察 kernel name、PTX 或编译产物，理解机器在执行什么。
4. **Change one thing**: 只改变一个实现维度，例如 fusion、block size、tile shape、数据布局。
5. **Re-benchmark and re-profile**: 性能工作必须闭环，否则只是猜谜。

本讲的核心观点

高性能 GPU 编程不是“把 Python 换成 CUDA”这么简单。正确性主要来自编程模型，例如 PyTorch、Triton、CUDA；性能则来自硬件事实，例如 warp 同步执行、shared memory bank、HBM coalescing、occupancy 和 tile 复用。真正可靠的优化路径，是把编程模型和硬件模型用 benchmark/profile 连接起来。

1.1 为什么 kernel 是 LLM 系统的基本单位

在 LLM 训练中，我们习惯从 layer、operator、tensor shape 看问题，例如 attention、MLP、layernorm、all-reduce。但 GPU 真正执行的是 kernel。一个 PyTorch expression 可能展开成多个 kernel，也可能被编译器融合成一个 kernel；一个高层算子可能调用 cuBLAS/CUTLASS 的巨大 matmul kernel，也可能退化成一个形状不友好的 fallback kernel。换句话说，operator 是用户看到的单位，kernel 是硬件看到的单位。

operator 与 kernel 的差异

`x * torch.sigmoid(x)` 是一个 operator-level expression，但朴素执行时可能包含乘法、sigmoid、临时张量写回等多个 kernel。融合之后，它可以变成一个 kernel：从 HBM 读一次 `x`，在寄存器里完成所有逐元素计算，再写一次结果。两者数学相同，内存账本完全不同。

1.2 本讲四个案例

官方讲义用四类 kernel 贯穿全讲。它们覆盖了 LLM 中最常见的性能形态。

案例	计算形态	讲义中要学的判断
GeLU	逐元素函数	为什么 naive PyTorch 会产生多次 HBM 读写；builtin、torch.compile 和 Triton 如何融合。
Softmax	行内 reduction	为什么稳定 softmax 需要 max/sum；当一整行能放进一个 block 时，怎样做到一次读写。
Row sum	跨 tile reduction	当一行太长放不进一个 block 时，怎样循环 tiles 并维护 accumulator。
Matmul + ReLU	tilted matrix multiply	为什么 tiling 是提高 arithmetic intensity 的核心，为什么 activation fusion 几乎是白捡收益。

表 1: Lecture 6 的四个 kernel 案例与对应性能问题。

1.3 本章小结

本讲不是 Triton 语法速查，而是一门“小型 GPU 性能实验课”。如果只看代码，不看 benchmark/profile，就会错过为什么代码快；如果只看硬件，不写 kernel，就会停留在抽象概念。讲义的目标是把两边接起来。

2 GPU 编程模型复习：正确性抽象与性能现实

2.1 从 grid 到 thread block

CUDA 的编程模型把一次 kernel launch 表示成一个 grid，grid 由很多 thread blocks 组成，每个 block 内有许多 threads。一个 block 会被调度到一个 SM 上，block 内 threads 可以通过 shared memory 协作；不同 block 之间通常不能直接共享低延迟状态，只能通过 global memory 或更高层同步。

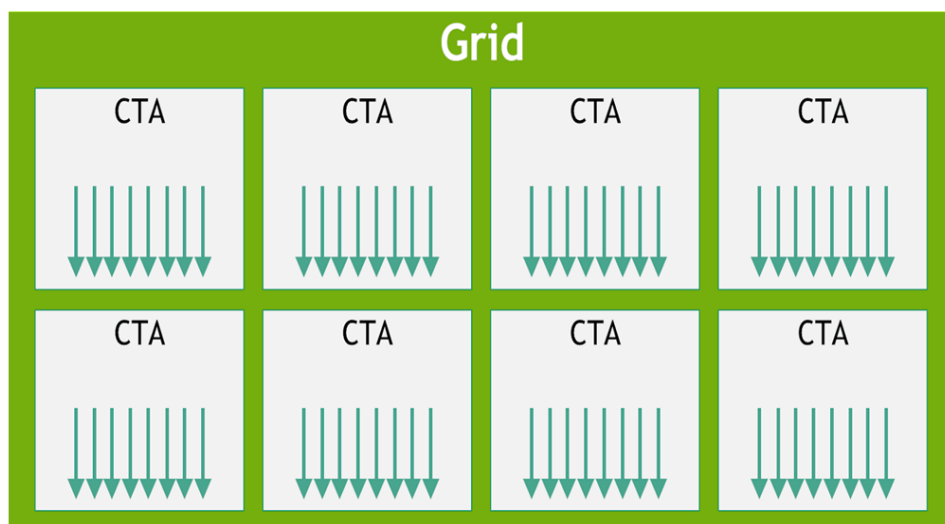
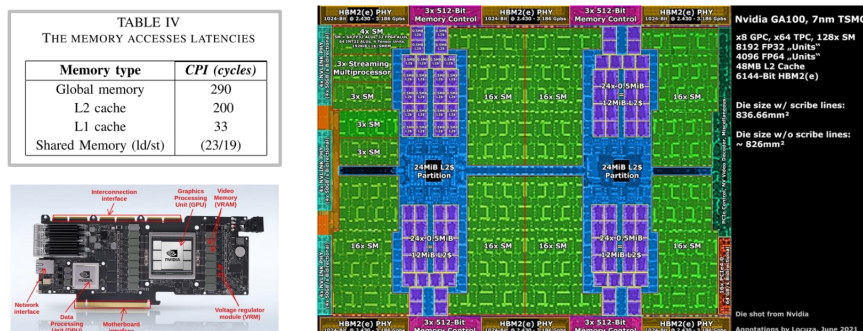


图 2: CUDA 编程模型：grid 包含多个 CTA/thread blocks，block 内 threads 共享局部资源。

来源：NVIDIA PTX 文档示意图，CS336 Spring 2026 Lecture 6 引用。

Anatomy of a GPU (memory)

The closer the memory to the SM, the faster it is – L1 and shared memory is *inside* the SM. L2 cache is on die, and global memory are the memory chips next to the GPU



SRAM (shared/cache memory) is much more expensive (100x) but ~ 8x faster than DRAM (Global memory)

图 3: 上一讲的 GPU 执行模型回顾：threads、blocks、warps 与 SM 的映射关系。

来源：CS336 Spring 2026 Lecture 5 官方幻灯片节选。

这个抽象足够让程序员写出正确程序，但不足以自动得到高性能。性能敏感点往往藏在抽象背

后: warp 内分支是否一致, global memory 访问能否 coalesce, shared memory 是否发生 bank conflict, 一个 block 使用多少 registers, grid 中 block 数是否让最后一波 SM 闲置。

编程模型和硬件模型的分工

编程模型回答“我可以怎样表达并行计算”；硬件模型回答“这个表达会不会让机器空转”。很多 LLM kernel 的优化，都是在不改变数学表达的前提下，让表达方式更贴近硬件吞吐模型。

2.2 Warp: 32 个线程同步前进

典型 NVIDIA GPU 中，一个 warp 含 32 个 threads。同一个 warp 内的 threads 以 SIMT 方式执行：它们共享指令流，但处理不同数据。如果 threads 走不同分支，硬件必须串行执行分支路径，并在每条路径上屏蔽不活跃 threads。这就是 control divergence。

Listing 1: warp divergence 的典型形态

```
1 int tid = blockIdx.x * blockDim.x + threadIdx.x;
2 if (tid % 2 == 0) {
3     y[tid] = expensive_path_a(x[tid]);
4 } else {
5     y[tid] = expensive_path_b(x[tid]);
6 }
```

这段代码在 CPU 上只是普通分支；在 GPU 上，如果同一个 warp 内奇偶线程交错出现，就会导致两条路径先后执行。LLM 中最常见的 divergence 来源包括 mask、边界处理、稀疏路由、变长序列以及 shape 不能整除 tile。

分支不是原罪，未对齐的分支才危险

GPU kernel 里经常需要 mask，例如处理最后一个不满 block 的 tile。mask 本身可以接受，因为它只影响少量边界元素；真正危险的是在主体计算区域让同一个 warp 中大量线程持续走不同路径。

2.3 Occupancy: 更多 warp 不一定等于更快

一个 SM 可以同时驻留多个 warps。当某个 warp 等待 HBM 读写时，SM 可以切换到另一个 ready warp，这让 GPU 用大量并发隐藏内存延迟。Occupancy 通常指一个 SM 上实际驻留 warps 与硬件上限 warps 的比例。

官方讲义给出的寄存器约束可以写成：

$$\text{blocks per SM} = \left\lfloor \frac{R_{\text{SM}}}{T_{\text{block}} \cdot R_{\text{thread}}} \right\rfloor, \quad \text{occupancy} = \frac{\text{blocks per SM} \cdot T_{\text{block}} / 32}{W_{\text{max}}}.$$

其中 R_{SM} 是一个 SM 可用寄存器数， T_{block} 是每个 block 的线程数， R_{thread} 是每个 thread 使用的寄存器数， W_{max} 是最大驻留 warp 数。寄存器用得越多，一个 SM 能同时驻留的 blocks/warps 越少。

Listing 2: Lecture 6 中 occupancy 账本的简化版本

```
1 num_threads_per_block = 128
2 num_registers_per_thread = 160
3
4 max_registers = 65536
5 max_warps = 64
6
7 num_registers_per_block = (
8     num_threads_per_block * num_registers_per_thread
9 )
10 num_blocks = max_registers // num_registers_per_block
11 num_warps = num_blocks * num_threads_per_block / 32
12 occupancy = num_warps / max_warps
```

低 occupancy 并不自动表示慢

如果每个 thread 做更多有用工作，例如 thread coarsening，一次 HBM 读取后在寄存器里完成更多计算，那么低一点的 occupancy 可能换来更高 arithmetic intensity。Occupancy 是诊断信号，不是优化目标本身。

2.4 Shared memory bank conflict

Shared memory 被切成多个 banks。常见设定是 32 个 banks，每个 bank 每个周期服务一个访问。如果一个 warp 的多个 threads 访问不同地址但落在同一个 bank，就会发生 bank conflict，访问被串行化。矩阵乘法中，按行访问 A、按列访问 B 很容易触发这种冲突，尤其当布局与 bank 映射形成固定步长时。

shared memory 不是免费缓存

把数据搬到 shared memory 只是第一步。若访问模式造成 bank conflicts，shared memory 的低延迟优势会被串行化抵消。高性能 GEMM kernel 里常见的 swizzling、padding、layout transform，本质上是在改变 bank 映射。

2.5 HBM coalescing 与 block occupancy

Global memory/HBM 的访问以 cache line 或 transaction 为单位合并。一个 warp 的 32 个 threads 如果访问连续的 32 个 float32，刚好是 128 bytes，通常可以形成理想 coalescing；如果访问跨步很大或随机地址，就会变成多个 transaction，浪费带宽。

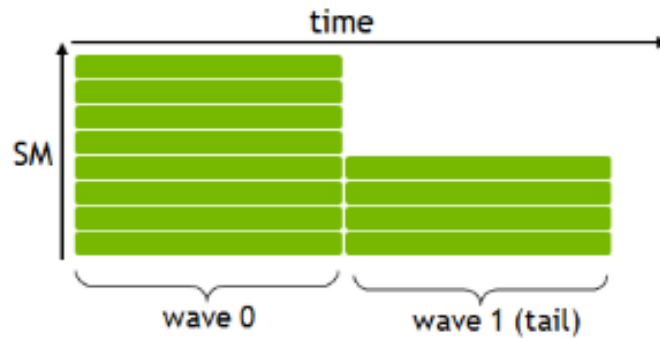


图 4: Block occupancy 的 wave quantization: 当 block 数不能填满最后一波 SM, 部分 SM 会闲置。

来源: NVIDIA Developer Blog 图示, CS336 Spring 2026 Lecture 6 引用。

另一个宏观问题是 block occupancy。假设一块 B200 有 148 个 SM, 如果 kernel 只 launch 160 个 blocks, 第一波可以填满 148 个 SM, 第二波只剩 12 个 blocks, 其余 136 个 SM 空闲。这类 wave quantization 在小 batch、小矩阵、短序列场景中尤其明显。

2.6 本章小结

GPU 编程模型给出了 grid、block、thread 的正确性抽象; 性能则由 warp divergence、register pressure、bank conflict、coalescing 和 block wave 这些硬件细节共同决定。Lecture 6 后面的 benchmark/profile 与 Triton 代码, 都是为了把这些细节变成可测、可改的工程对象。

3 Benchmarking: GPU 计时首先要可信

3.1 为什么 CPU wall-clock 容易误导

CUDA kernel launch 对 CPU 来说通常是异步的。Python 调用返回时, GPU 可能还在排队或执行。如果直接用 `time.time()` 包住一个 CUDA 操作, 测到的可能主要是 CPU 发起 kernel 的开销, 而不是 GPU 真正执行的时间。正确计时需要显式同步, 或者使用 CUDA events。

Benchmark 回答的问题

Benchmark 只回答端到端时间, 例如“这个函数平均多少毫秒”。它不回答时间花在哪里, 也不解释为什么。解释原因要交给 profiler。

官方讲义展示了一个透明的 benchmark 函数。它有三个关键动作: warmup、CUDA event 计时、同步。

Listing 3: 使用 CUDA events 的 GPU benchmark 模板

```

1 def benchmark(run, num_warmups=1, num_trials=3):
2     # Warmup: first runs can include compilation and cache effects.
3     for _ in range(num_warmups):
4         run()
5     torch.cuda.synchronize()
6
7     times = []
8     for _ in range(num_trials):
9         start_event = torch.cuda.Event(enable_timing=True)
10        end_event = torch.cuda.Event(enable_timing=True)
11
12        start_event.record()
13        run()
14        end_event.record()
15
16        torch.cuda.synchronize()
17        times.append(start_event.elapsed_time(end_event))
18
19    return sum(times) / len(times)

```

3.2 Warmup 的真实含义

Warmup 不只是“让机器热起来”。在现代 PyTorch 工作流中，第一次运行可能包含 lazy initialization、allocator 初始化、cuBLAS heuristic 选择、Triton 编译、`torch.compile` 图捕获和代码生成。若把第一次运行计入 steady-state 速度，往往会夸大延迟。

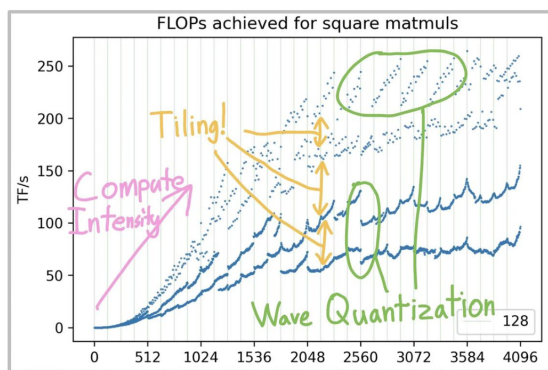
不要把 compile time 和 runtime 混在一起

`torch.compile` 和 Triton kernel 第一次调用经常包含编译成本。训练中如果 shape 固定，这个成本可以摊销；在线推理中如果 shape 多变，编译缓存命中率会影响真实 latency。benchmark 时要明确自己测的是 cold start 还是 steady state。

3.3 维度扫描比单点计时更有价值

单个 shape 的计时经常无法解释扩展规律。官方讲义对 matmul 扫描 256, 512, 1024, ..., 8192 这样的维度，就是为了观察从 launch overhead/underutilization 区间进入 cubic scaling 区间的过程。小矩阵常常不是 FLOPs 不够快，而是没有足够工作填满 GPU；大矩阵则更接近计算或内存 rooeline。

Part 2: Making ML workloads fast on a GPU



Performance on a GPU can be complex, even for something as simple as a square matmul

图 5: Roofline 视角：同一个算子在不同 shape 下可能落在 launch/内存/计算三个不同瓶颈区域。

来源：CS336 Spring 2026 Lecture 5 官方幻灯片节选。

benchmark 设计要像小实验

一个好的 benchmark 不只是输出一个数字，而是回答一个假设。例如：“当 N 增大时，matmul 是否从 launch-bound 变成 compute-bound?” 或者 “fusion 是否真的把 HBM traffic 降到一次读写?” 如果没有假设，就很难从数字中学到东西。

3.4 常见计时陷阱

陷阱	表现	处理方式
未同步	时间过小，只测到 CPU launch	使用 CUDA events 或 <code>torch.cuda.synchronize()</code> 。
只测 cold start	<code>torch.compile</code> /Triton 看起来很慢	区分首次编译时间和 steady-state 时间。
输入包含分配成本	算子时间混入 <code>torch.randn</code> /allocator 开销	在 closure 外创建输入，只在 closure 内运行待测操作。
shape 太少	得到孤立数字，无法判断扩展趋势	扫描 batch、sequence、hidden、tile-friendly 与非友好 shape。
忽略 variance	单次测量受调度和缓存影响	多次 trials，报告均值和必要时的分布。

3.5 本章小结

Benchmark 的第一要求不是花哨，而是可信。对 GPU 而言，可信意味着明确同步、区分 warmup 与 steady state、把输入准备排除在外，并用维度扫描观察 scaling 行为。后续所有优化都要回到这个闭环。

4 Profiling: 从 PyTorch 表达式看到 CUDA kernel

4.1 Profiler 解决 benchmark 解决不了的问题

Benchmark 告诉你某段代码用了多少时间, 但不知道时间花在哪。Profiler 则展示 PyTorch 背后实际执行的 CUDA kernels、每个 kernel 的 CUDA time、调用次数和占比。对 LLM 工程而言, profile 的最大价值是打破“高层 API 名称”的幻觉: 你写的是 `a @ b`, 机器执行的可能是某个 `cuBLASLt`、`CUTLASS` 或 `fallback kernel`。

Listing 4: 使用 torch.profiler 观察 CUDA kernel

```
1 from torch.profiler import ProfilerActivity
2
3 def profile(run, num_warmups=1):
4     for _ in range(num_warmups):
5         run()
6     torch.cuda.synchronize()
7
8     with torch.profiler.profile(
9         activities=[ProfilerActivity.CUDA],
10        experimental_config=torch._C._profiler._ExperimentalConfig(
11            verbose=True,
12        ),
13    ) as prof:
14        run()
15        torch.cuda.synchronize()
16
17    return prof.key_averages().table(
18        sort_by="cuda_time_total",
19        max_name_column_width=100,
20        row_limit=10,
21    )
```

4.2 Kernel name 是性能线索

官方讲义中特别提到 kernel name 里可能出现类似:

`cutlass3x_sm100_simt_sgemm_f32_..._64x64x16_...`

这串名字虽然长, 却携带大量信息: `cutlass` 表示 NVIDIA 的 CUDA linear algebra library; `sm100` 指向 Blackwell/B200 架构; `sgemm` 表示 float32 GEMM; `64x64x16` 暗示 tile shape。Profile 不是只看总时间, 还要学会从这些名字推断实现路径。

profile 的阅读顺序

先看是否有少数 kernel 占据绝大部分时间; 再看 kernel 数量是否异常多; 然后看 kernel name 是否符合预期库路径; 最后结合 shape 和 dtype 判断是否落入了高效 kernel。这个顺序比一开始就读 PTX 更有效。

4.3 add、matmul 和 shape 依赖

逐元素 add 通常是 memory-bound：每个元素读两个输入、写一个输出，计算只有一次加法。Matmul 则可能 compute-bound，也可能因为 shape 太小、tile 不友好或 dtype 不合适而利用率很差。官方讲义 profile add(dim=2048)、matmul(dim=2048) 和 matmul(dim=128)，目的就是让学生看到同一个高层操作在不同 shape 下会选择不同 kernel，性能含义也不同。

小 shape 常常慢在“没把 GPU 喂饱”

当矩阵很小或 batch 很小，GPU 的大量 SM 无法同时获得足够 work。此时总 FLOPs 很少，理论峰值毫无意义；kernel launch、wave quantization、tile padding 和内存延迟都可能主导 latency。

4.4 Profiler 与 Nsight 的关系

torch.profiler 足以回答“PyTorch 调了哪些 kernel、各占多少时间”。作业中进一步使用 Nsight，是为了看到更底层指标，例如 achieved occupancy、memory throughput、SM utilization、warp stalls、L2 hit rate、shared memory bank conflicts。两者不是替代关系，而是分层诊断。

不要跳过 PyTorch profiler

很多性能问题在 PyTorch profiler 层已经很明显：kernel 数太多、某个 fallback kernel 异常慢、shape 触发了非预期实现。直接上 Nsight 可能会带来大量底层指标，却忘了先确认高层 execution plan 是否合理。

4.5 本章小结

Profile 把高层 tensor 程序翻译成硬件执行痕迹。读懂 profiler 表格，是从“我觉得这里慢”走向“证据显示这个 kernel 慢”的关键一步。接下来的 GeLU 案例会展示为什么 kernel 数量和 HBM traffic 会直接改变性能。

5 GeLU：逐元素函数里的 fusion 账本

5.1 GeLU 公式与 naive PyTorch

GeLU 是 Transformer MLP 中常见的激活函数。课程使用 tanh approximation：

$$\text{GeLU}(x) = \frac{1}{2}x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right).$$

在 PyTorch 中直接按公式写很自然：

Listing 5: naive GeLU: 表达简单，但会生成多个中间结果

```

1 def naive_gelu(x):
2     return 0.5 * x * (
3         1 + torch.tanh(
4             0.79788456 * (x + 0.044715 * x * x * x)
5         )
6     )

```

数学上这段代码没有问题，性能上却会暴露 PyTorch eager execution 的典型成本：每个逐元素子表达式都可能产生独立 kernel 和临时张量，中间结果反复读写 HBM。

5.2 内存 traffic 为什么会压过 FLOPs

GeLU 的每个元素计算包含乘法、加法、指数或 tanh 等操作。看起来 FLOPs 不少，但相对现代 GPU 的计算峰值，逐元素函数经常仍然是 memory-bound。原因是每个中间张量都要写回 HBM，再被下一个 kernel 读入。若一个 naive expression 产生 5 个 kernel，就可能对同一份数据进行多轮读写。

实现	kernel 数量	主要成本
naive PyTorch	多个逐元素 kernel	中间结果多次写回 HBM，kernel launch 多。
PyTorch builtin	通常单个融合 kernel	一次读入，一次写出，库实现处理细节。
torch.compile	编译生成融合 kernel	从高层 Python 捕获图，再下沉到优化 kernel。
Triton	用户显式写 block-level kernel	控制 load/compute/store 和 mask，便于实验。

表 3: GeLU 几种实现方式的主要区别。

Fusion 的本质

Fusion 不是减少 GeLU 的数学计算，而是减少临时结果离开寄存器、写回 HBM、再读回来的次数。逐元素函数的优化经常不在公式本身，而在中间结果是否留在片上。

5.3 builtin 与 torch.compile

PyTorch 的 builtin GeLU 已经是经过优化的实现：

Listing 6: PyTorch builtin GeLU

```

1 def builtin_gelu(x):
2     return torch.nn.functional.gelu(x, approximate="tanh")

```

`torch.compile` 则展示了现代 compiler stack 的方向：用户仍然写 naive Python，但编译器尝试捕获计算图，融合逐元素操作，生成更少 kernel。官方讲义指出 compiled kernel 是 Triton kernel，这说明 Triton 不只是手写 kernel 工具，也是 PyTorch 编译后端的重要组成部分。

Listing 7: 用 `torch.compile` 融合 naive GeLU

```

1 compiled_gelu = torch.compile(naive_gelu)
2 y = compiled_gelu(x)

```

何时相信 compiler，何时手写

对常见 pointwise fusion，优先相信 `torch.compile` 或库 builtin。手写 Triton 的理由通常是：需要特殊数据布局、特殊 reduction、特殊 masking、和多个算子跨边界融合，或者 profiler 证明 compiler 没有生成理想 kernel。

5.4 本章小结

GeLU 案例传达的不是“手写一定更快”，而是“先看 kernel 数和 HBM traffic”。在很多逐元素算子中，性能差距来自 fusion，而 fusion 的收益可以由 profiler 直接验证：naive 多个 kernel，builtin/compiled 单个 kernel。

6 Triton 编程模型：在 thread block 层写 GPU 程序

6.1 CUDA 与 Triton 的抽象边界

CUDA C++ 通常让程序员指定每个 thread 做什么，控制粒度非常细；Triton 则让程序员更自然地指定一个 program instance，也可以理解成一个 block/tile 做什么。Triton 程序用向量化的 `tl.arange`、`tl.load`、`tl.store`、`tl.sum`、`tl.dot` 表达 block 内计算，编译器负责把它 lowering 到 PTX 和更底层指令。

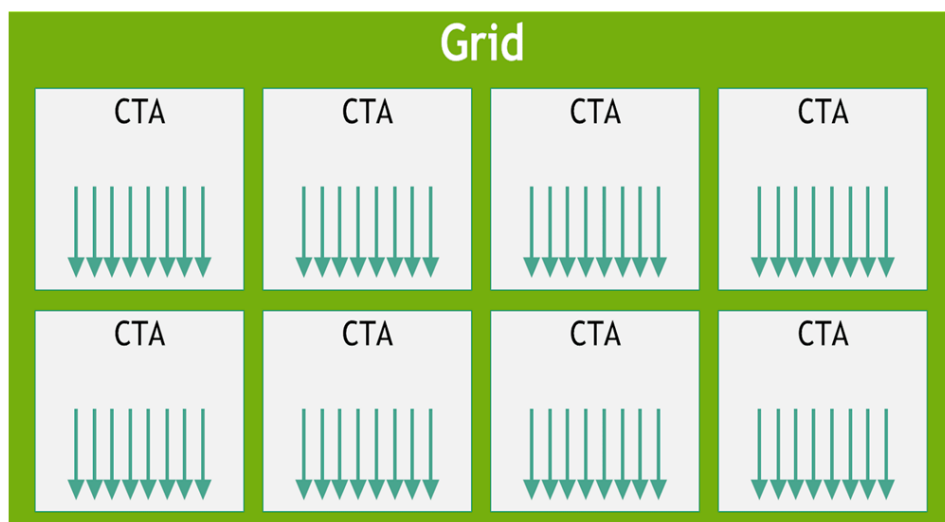


图 6: Triton 的 program instance 可以粗略类比为 CUDA block/CTA 层抽象：每个 program 处理一个 tile。

来源：NVIDIA PTX 文档示意图，CS336 Spring 2026 Lecture 6 引用。

Triton 的心智模型

把一个 tile 从 HBM load 到片上，在片上完成尽可能多的计算和融合，再 store 回 HBM。这个模型比“每个 thread 写什么”更接近算法设计，又比纯 PyTorch 更接近硬件。

6.2 一个 Triton kernel 的骨架

下面是讲义中 Triton 代码反复出现的骨架。

Listing 8: Triton kernel 的常见骨架

```

1 @triton.jit
2 def kernel(x_ptr, y_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
3     pid = tl.program_id(axis=0)
4     offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
5     mask = offsets < n_elements
6
7     x = tl.load(x_ptr + offsets, mask=mask)
8     y = some_vectorized_compute(x)
9     tl.store(y_ptr + offsets, y, mask=mask)

```

几个概念值得单独记住：

- `tl.program_id(axis=0)` 给出当前 program/block 的编号。
- `tl.arange(0, BLOCK_SIZE)` 生成 block 内向量 offsets。
- `mask` 处理越界，尤其是最后一个不满 block 的 tile。
- `tl.constexpr` 表示编译期常量，允许编译器据此展开和优化。

6.3 Host wrapper 与 grid

Triton kernel 不是直接像普通 Python 函数一样调用，而是通过 launch grid 调用。host wrapper 通常负责检查输入、分配输出、计算 grid shape、传入 strides 和 compile-time block sizes。

Listing 9: host wrapper 负责分配输出与 launch grid

```
1 def triton_op(x):
2     assert x.is_cuda
3     assert x.is_contiguous()
4
5     y = torch.empty_like(x)
6     n = x.numel()
7     block_size = 1024
8     grid = (triton.cdiv(n, block_size),)
9
10    kernel[grid](x, y, n, BLOCK_SIZE=block_size)
11    return y
```

Triton 不是自动正确

Triton 让写 kernel 更容易，但不会替你证明 shape、stride、dtype、mask 都正确。高性能 kernel 的第一要求仍然是 correctness: contiguous 假设、stride 计算、越界 mask、数值稳定性和 dtype accumulation 都要明确。

6.4 从 Triton 到 PTX

官方讲义在 GeLU 例子中把 Triton 生成的 PTX 写出到文件，并观察：

- `ld.global.*` 和 `st.global.*` 对应 global memory 读写；
- `%ctaid.x` 类似 block id；
- `%tid.x` 类似 thread id；
- `%f*` 是浮点寄存器，`%r*` 是整数寄存器；
- 一个 thread 可能一次处理多个元素，这就是 thread coarsening。

读 PTX 的目的

多数时候不需要手工优化 PTX。读 PTX 的价值是验证 mental model: 有没有 global load/store, 临时变量是否主要在 registers, 是否发生预期的 thread coarsening。它是 profile 之后的更底层证据，而不是日常第一工具。

6.5 本章小结

Triton 站在 PyTorch 和 CUDA C++ 中间：比 PyTorch 更明确地控制 tile、mask、load/store 和 fusion，比 CUDA 少管理很多 thread-level 细节。Lecture 6 的后半部分，就是用这个抽象逐步写出 elementwise、reduction 和 tiled matmul kernel。

7 Triton GeLU: 最小但完整的融合 kernel

7.1 Launch 逻辑

GeLU 是最简单的 Triton 例子，因为每个元素可以独立计算。host side 做三件事：分配输出、把元素数除以 block size 得到 block 数、launch kernel。

Listing 10: Triton GeLU 的 host wrapper

```
1 def triton_gelu(x):
2     assert x.is_cuda
3     assert x.is_contiguous()
4
5     y = torch.empty_like(x)
6     num_elements = x.numel()
7     block_size = 1024
8     num_blocks = triton.cdiv(num_elements, block_size)
9
10    triton_gelu_kernel[(num_blocks,)](
11        x, y, num_elements, BLOCK_SIZE=block_size
12    )
13    return y
```

7.2 Kernel 逻辑

Listing 11: Triton GeLU kernel: 一次 load, 寄存器内计算, 一次 store

```
1 @triton.jit
2 def triton_gelu_kernel(x_ptr, y_ptr, num_elements,
3                        BLOCK_SIZE: tl.constexpr):
4     pid = tl.program_id(axis=0)
5     start = pid * BLOCK_SIZE
6     offsets = start + tl.arange(0, BLOCK_SIZE)
7     mask = offsets < num_elements
8
9     x = tl.load(x_ptr + offsets, mask=mask)
10
11    a = 0.79788456 * (x + 0.044715 * x * x * x)
12    exp = tl.exp(2 * a)
13    tanh = (exp - 1) / (exp + 1)
14    y = 0.5 * x * (1 + tanh)
15
16    tl.store(y_ptr + offsets, y, mask=mask)
```

这里的性能收益来自两个层面。第一，所有中间变量 `a`、`exp`、`tanh`、`y` 都在 block 内向量表达式中生成，理想情况下主要驻留在 registers，而不是写成临时 HBM 张量。第二，每个元素只需要从 HBM 读一次、写一次，避免 naive PyTorch 多 kernel 的中间 traffic。

GeLU kernel 的最小账本

理想 fused GeLU 的 HBM traffic 是 N 个输入读和 N 个输出写。naive eager GeLU 的数学表达虽然一样，但每个子表达式都可能变成新的 HBM 读写。对 memory-bound pointwise 算子，traffic 差异就是速度差异。

7.3 Mask 的语义

如果 N 不是 BLOCK_SIZE 的整数倍，最后一个 block 的部分 offsets 会越界。Triton 的 `mask` 让 `tl.load` 和 `tl.store` 只处理有效元素。这一点在所有真实 kernel 中都很重要，因为 LLM shape 经常不是 tile size 的整数倍。

边界 mask 是 correctness 逻辑，不是可选优化

为了追求速度而省略 mask，只有在你能严格证明 shape 总是整除 block/tile 时才成立。通用 kernel 通常必须处理尾部 tile，否则会读写越界。

7.4 本章小结

Triton GeLU 是一个入门 kernel，但已经包含 GPU kernel 的核心骨架：grid 切分、program id、offsets、mask、load、寄存器内计算、store。理解这个例子之后，softmax 和 matmul 的区别主要在于是否需要 block 内 reduction 和 tile 间复用。

8 Fused Softmax：当一整行放进一个 block

8.1 Softmax 的稳定形式

Softmax 对每一行独立计算：

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}.$$

为了数值稳定，实际实现通常先减去行最大值：

$$m = \max_j x_j, \quad y_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}.$$

这不会改变 softmax 结果，因为分子分母同时乘以 $\exp(-m)$ ，但可以避免大正数指数溢出。

8.2 Naive softmax 的 HBM 账本

官方讲义中的 naive PyTorch softmax 拆成五步：行 max、减 max、exp、行 sum、normalize。若矩阵是 $M \times N$ ，粗略账本是：

$$\text{reads} \approx 5MN + M, \quad \text{writes} \approx 3MN + 2M.$$

而理想 fused softmax 只需要读一次 MN ，写一次 MN ，中间 max、sum、numerator 都留在 block 内。

Listing 12: naive softmax 的分步表达

```

1 def naive_softmax(x):
2     M, N = x.shape
3     x_max = x.max(dim=1)[0]
4     x = x - x_max[:, None]
5     numerator = torch.exp(x)
6     denominator = numerator.sum(dim=1)
7     y = numerator / denominator[:, None]
8     return y

```

Softmax fusion 的特殊性

GeLU 只需要逐元素融合；softmax 还需要 reduction。只要一整行能放进一个 Triton block，max 和 sum 都可以在 block 内完成，从而避免把中间矩阵写回 HBM。

8.3 Triton fused softmax

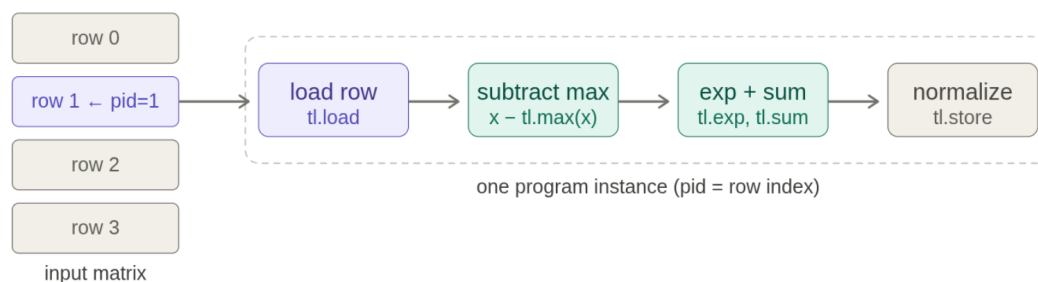


图 7: Triton fused softmax: 每个 program 处理一行，行内做 max、exp、sum 和 normalize。

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义配图。

Triton softmax 的 grid 很自然：矩阵有 M 行，就 launch M 个 programs；每个 program 负责一行。为了让 block 内向量长度适合编译器，常用 `triton.next_power_of_2(N)` 作为 block size，并对越界列用 mask。

Listing 13: Triton softmax host wrapper

```

1 def triton_softmax(x):
2     y = torch.empty_like(x)
3     M, N = x.shape
4     block_size = triton.next_power_of_2(N)
5
6     triton_softmax_kernel[(M,)](
7         x_ptr=x,
8         y_ptr=y,
9         x_row_stride=x.stride(0),
10        y_row_stride=y.stride(0),
11        num_cols=N,
12        BLOCK_SIZE=block_size,
13    )
14    return y

```

Listing 14: Triton fused softmax kernel

```

1 @triton.jit
2 def triton_softmax_kernel(x_ptr, y_ptr,
3                           x_row_stride, y_row_stride,
4                           num_cols,
5                           BLOCK_SIZE: tl.constexpr):
6     row_idx = tl.program_id(0)
7     col_offsets = tl.arange(0, BLOCK_SIZE)
8
9     x_start_ptr = x_ptr + row_idx * x_row_stride
10    x_ptrs = x_start_ptr + col_offsets
11    x_row = tl.load(
12        x_ptrs,
13        mask=col_offsets < num_cols,
14        other=float("-inf"),
15    )
16
17    x_row = x_row - tl.max(x_row, axis=0)
18    numerator = tl.exp(x_row)
19    denominator = tl.sum(numerator, axis=0)
20    y_row = numerator / denominator
21
22    y_start_ptr = y_ptr + row_idx * y_row_stride
23    y_ptrs = y_start_ptr + col_offsets
24    tl.store(y_ptrs, y_row, mask=col_offsets < num_cols)

```

8.4 为什么越界值用负无穷

对 softmax 来说，越界列不能贡献 max 或 sum。用 `other=float("-inf")` 读取越界位置有两个好处：max reduction 不会受影响，随后 $\exp(-\infty) = 0$ ，sum reduction 也不会受影响。这个处理方式比随便填 0 更安全，因为如果真实行元素都是负数，越界 0 可能错误地成为最大值。

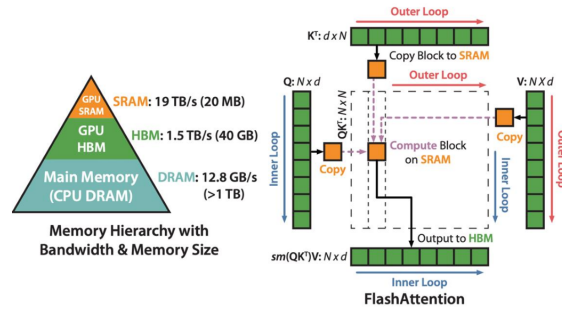
mask 的填充值必须符合算子语义

对 sum，越界值常填 0；对 max，越界值应填 $-\infty$ ；对 min，越界值应填 $+\infty$ 。同样是 mask，不同 reduction 的 neutral element 不同。

8.5 行能否放进 block 是关键假设

上面的 fused softmax 依赖一个重要假设：一整行长度 N 可以放进一个 block 的向量寄存器和片上资源。如果 N 很大，block size 过大可能导致 register pressure 爆炸、occupancy 下降，甚至超过 Triton/硬件限制。此时需要把一行切成多个 tiles，用 online softmax 或多阶段 reduction。

Tiling part 1: tiling for the KQV matrix multiply



This figure 1 from the paper is literally just tiling for a KQV matrix multiply..

But what do we do about the softmax?

图 8: online softmax 思想：用流式 max/sum 状态支持 tile-by-tile 处理长序列。

来源：CS336 Spring 2026 Lecture 5 官方幻灯片节选。

8.6 本章小结

Softmax 把 Lecture 6 从 pointwise fusion 推向 reduction fusion。当一整行能放进一个 block, Triton 可以在单个 kernel 内完成 max、exp、sum、normalize, 把 HBM traffic 从多轮中间读写降到接近一次读写。长行场景则需要下一节的跨 tile reduction 思路。

9 Row Sum: 当一行放不进一个 block

9.1 从 softmax 简化到 row sum

如果 softmax 的一行太长, 不能一次放进一个 block, 我们需要把行切成多个 tiles, 逐 tile 累积必要状态。官方讲义先用 row sum 作为简化案例, 因为 row sum 只有一个 accumulator, 不需要同时维护 max 和 rescaled sum。

$$y_m = \sum_{n=0}^{N-1} x_{m,n}.$$

当 BLOCK_SIZE=1024 而 $N = 4096$, 一个 program 可以对同一行循环四个 tiles, 每次 load 1024 个元素, 加到 accumulator, 最后对 accumulator 做 block 内 sum。

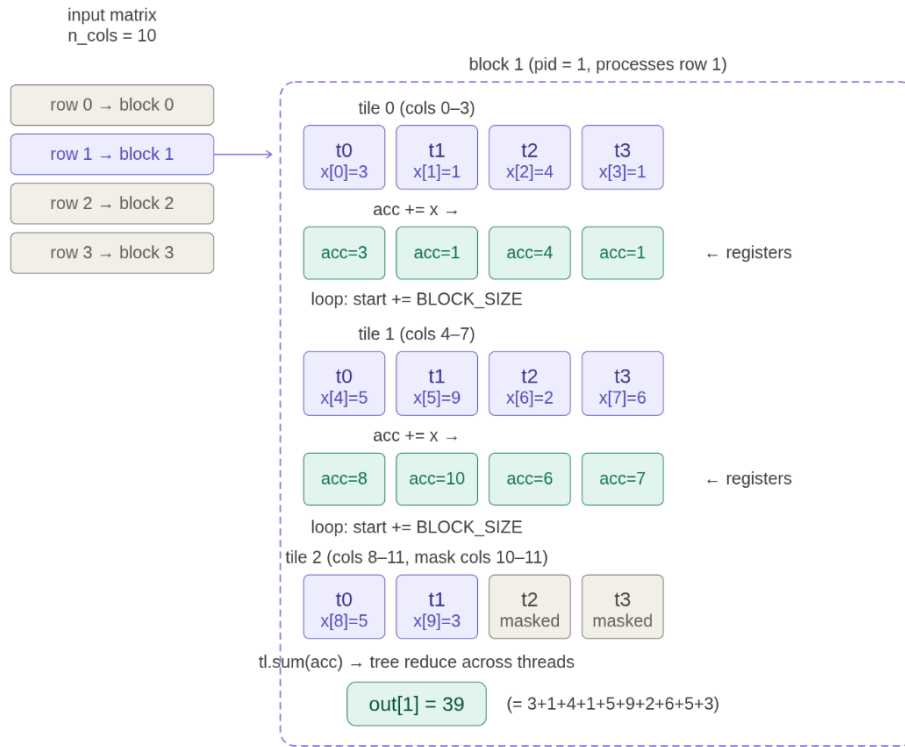


图 9: Row sum 跨 tile reduction: 同一行被切成多个 tile, block 内维护 accumulator。

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义配图。

9.2 Triton row sum kernel

Listing 15: 跨 tile row sum kernel

```

1 @triton.jit
2 def row_sum_kernel(x_ptr, out_ptr, N,
3                     BLOCK_SIZE: tl.constexpr):
4     row = tl.program_id(0)
5     acc = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
6
7     for start in range(0, N, BLOCK_SIZE):
8         cols = start + tl.arange(0, BLOCK_SIZE)
9         mask = cols < N
10        x = tl.load(
11            x_ptr + row * N + cols,
12            mask=mask,
13            other=0.0,
14        )
15        acc += x
16
17    result = tl.sum(acc, axis=0)
18    tl.store(out_ptr + row, result)

```

这个例子看起来朴素, 却非常重要。它展示了 Triton program 内部的循环: 一个 program 不一定只处理一个 tile, 也可以沿着某个维度顺序扫多个 tiles, 把中间状态留在 registers 中。对 row

sum 来说，状态就是 `acc`；对 online softmax 来说，状态会变成当前 max 和 rescaled sum。

跨 tile reduction 的核心问题

当数据不能一次放入片上资源，算法必须决定保留什么状态。row sum 保留部分和；online softmax 保留 running max 和 running normalizer；FlashAttention 保留每个 query block 的局部输出、max 和 normalization 状态。

9.3 从 row sum 到 online softmax

Softmax 不能简单把每个 tile 的 $\exp(x)$ sum 相加，因为数值稳定需要全局 max。online softmax 的状态更新可以写成：

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}}),$$

$$\ell_{\text{new}} = \ell_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}) + \ell_{\text{tile}} \exp(m_{\text{tile}} - m_{\text{new}}).$$

这样即使分 tile 处理，也能保持与全行稳定 softmax 等价的 normalization。FlashAttention 的关键正是把这个 online softmax 状态和 QK/V 的 tiling 结合起来。

row sum 是 baby tiling

Row sum 是最简单的跨 tile reduction。它不需要重新缩放旧状态，因此容易理解。真正复杂的 attention/softmax kernel，只是在这个结构上增加数值稳定状态和更多矩阵乘法。

9.4 本章小结

当一行不能放进一个 block，kernel 设计从“单 block 内完成 reduction”变成“循环 tiles 并维护状态”。这一步是理解 FlashAttention、长序列 softmax、large-vocab logits reduction 的桥梁。

10 Matmul + ReLU: tiling、复用与融合

10.1 Naive matmul 的 arithmetic intensity

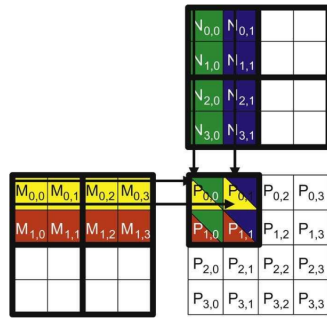
矩阵乘法 $C = AB$ ，其中 $A \in \mathbb{R}^{M \times K}$ ， $B \in \mathbb{R}^{K \times N}$ ， $C \in \mathbb{R}^{M \times N}$ 。naive 地看，每个 $C_{m,n}$ 都要读 K 个 $A_{m,k}$ 和 K 个 $B_{k,n}$ ，乘加得到一个输出：

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

如果不复用数据，HBM reads 近似 MKN 级别，arithmetic intensity 只有 $O(1)$ 。但理想情况下， A 的一个元素可以被同一行的多个 C 复用， B 的一个元素可以被同一列的多个 C 复用。Matmul 高性能的核心就是把这种复用搬到片上。

Tiling – store and reuse information in shared memory

Cut up the matrix into smaller ‘tiles’, and load this into shared memory



Compute the matrix multiply in ‘phases’

1. Load $M_{0,0}$ and $N_{0,0}$ tiles into SHM
2. Compute partial sums for P
(Done with one tile)
3. Load the $M_{0,0}$ and $N_{2,0}$ tile into SHM
4. ...

Advantages: repeated reads now access shared, not global memory
and memory access can be coalesced

图 10: 上一讲 tiling 数学回顾: tile 越能复用片上数据, arithmetic intensity 越高。

来源: CS336 Spring 2026 Lecture 5 官方幻灯片节选。

10.2 Tiling 的基本思路

理想化地说, 如果能把整个 A 和 B 都放进 shared memory, 就可以大幅减少 HBM reads。但真实矩阵太大, shared memory 很小, 所以只能 tile。一个 output tile $C_{m:m+B_M, n:n+B_N}$ 由多个 A tile 和 B tile 沿 K 维累加得到。

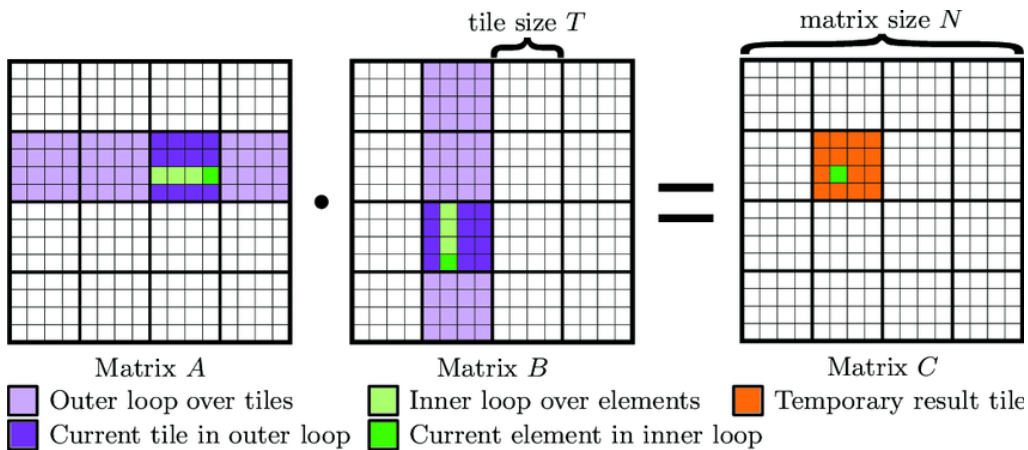


图 11: GEMM tiling: 每个 output tile 循环读取对应的 A tile 和 B tile, 在片上累加。

来源: CS336 Spring 2026 Lecture 6 官方可执行讲义配图。

如果 tile size 是 B , 一个 tile 里的数据会被复用 $O(B)$ 次, arithmetic intensity 也从 $O(1)$ 提升到 $O(B)$ 的量级。这正是 GEMM 能接近 GPU 计算峰值的原因, 也是 LLM 中 hidden size、batch size、sequence length 的 shape 会影响吞吐的原因。

Matmul 快，是因为复用密度高

矩阵乘法之所以是 GPU 的“特权算子”，不是因为乘加本身神奇，而是因为它天然拥有高数据复用。Tensor Cores、shared memory tiling、L2 swizzling、warp-level MMA 都是在把这种复用转成实际吞吐。

10.3 Triton matmul + ReLU 的 launch

官方讲义的 Triton matmul+ReLU 使用 64×64 output tile 和 K 方向 32 tile:

Listing 16: Triton matmul+ReLU 的 host wrapper

```
1 def triton_matmul_relu(a, b):
2     assert a.is_cuda and b.is_cuda
3     assert a.is_contiguous() and b.is_contiguous()
4     assert a.shape[1] == b.shape[0]
5
6     M, K = a.shape
7     K, N = b.shape
8     c = torch.empty((M, N), device=a.device)
9
10    BLOCK_M, BLOCK_N, BLOCK_K = 64, 64, 32
11    grid = (triton.cdiv(M, BLOCK_M),
12            triton.cdiv(N, BLOCK_N))
13
14    matmul_relu_kernel[grid](
15        a, b, c, M, N, K,
16        a.stride(0), a.stride(1),
17        b.stride(0), b.stride(1),
18        c.stride(0), c.stride(1),
19        BLOCK_M, BLOCK_N, BLOCK_K,
20    )
21    return c
```

这里的 grid 是二维的：一个维度遍历 M 方向 output tiles，另一个维度遍历 N 方向 output tiles。每个 program 负责一个 $B_M \times B_N$ 的输出块。

10.4 Kernel 核心：指针矩阵与 K 循环

Listing 17: matmul+ReLU kernel 的核心结构

```

1 @triton.jit
2 def matmul_relu_kernel(
3     a_ptr, b_ptr, c_ptr,
4     M, N, K,
5     stride_am, stride_ak,
6     stride_bk, stride_bn,
7     stride_cm, stride_cn,
8     BLOCK_M: tl.constexpr,
9     BLOCK_N: tl.constexpr,
10    BLOCK_K: tl.constexpr,
11 ):
12     pid_m = tl.program_id(0)
13     pid_n = tl.program_id(1)
14
15     offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
16     offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
17     offs_k = tl.arange(0, BLOCK_K)
18
19     a_ptrs = a_ptr + offs_m[:, None] * stride_am \
20         + offs_k[None, :] * stride_ak
21     b_ptrs = b_ptr + offs_k[:, None] * stride_bk \
22         + offs_n[None, :] * stride_bn
23
24     acc = tl.zeros([BLOCK_M, BLOCK_N], dtype=tl.float32)
25
26     for k in range(0, K, BLOCK_K):
27         a = tl.load(
28             a_ptrs,
29             mask=(offs_m[:, None] < M)
30             & (offs_k[None, :] + k < K),
31             other=0.0,
32         )
33         b = tl.load(
34             b_ptrs,
35             mask=(offs_k[:, None] + k < K)
36             & (offs_n[None, :] < N),
37             other=0.0,
38         )
39         acc += tl.dot(a, b)
40         a_ptrs += BLOCK_K * stride_ak
41         b_ptrs += BLOCK_K * stride_bk
42
43     acc = tl.maximum(acc, 0.0)
44
45     c_ptrs = c_ptr + offs_m[:, None] * stride_cm \
46         + offs_n[None, :] * stride_cn
47     tl.store(
48         c_ptrs,
49         acc,

```

```

50     mask=(offs_m[:, None] < M) & (offs_n[None, :] < N),
51 )

```

这段代码的关键是指针矩阵。对于 A ，指针形状是 $B_M \times B_K$ ；对于 B ，指针形状是 $B_K \times B_N$ ；`tl.dot(a,b)` 产出 $B_M \times B_N$ 的局部累加。每次循环沿 K 方向推进一个 tile。

10.5 为什么 ReLU fusion 几乎免费

如果单独做 `relu(a @ b)`，流程可能是 matmul kernel 先把 C 写回 HBM，随后 ReLU kernel 再读 C 、写 C' 。在 fused kernel 中，`acc` 还在 registers/片上状态中，执行 `tl.maximum(acc, 0.0)` 后直接 store。多做的计算极少，省掉的 HBM read/write 和 kernel launch 很实在。

Activation fusion 的工程含义

很多 LLM MLP 的性能优化不是单独优化 GeLU，而是把 linear、bias、activation、dropout 或 residual 的某些部分融合到邻近 kernel。融合边界一旦跨过 HBM 写回点，收益就会明显。

10.6 真实 GEMM kernel 更复杂在哪里

讲义中的 Triton matmul 是教学版。生产级 GEMM 还会处理更多细节：Tensor Core 指令选择、warp group 调度、L2 tile ordering、shared memory swizzling、double buffering、异步拷贝、epilogue fusion、split-K、persistent kernel、不同 dtype accumulation。理解教学版的价值，是知道这些复杂技术都在服务同一个目标：用最少 HBM traffic 和最高片上复用喂满矩阵乘法硬件。

不要用教学 matmul 替代成熟库

除非 profiler 明确显示库实现无法处理你的特殊形状或融合需求，否则 GEMM 优先使用 cuBLAS/cuBLASLt/CUTLASS/PyTorch builtin。手写 matmul 是理解和特殊化的工具，不是默认生产选择。

10.7 本章小结

Matmul+ReLU 把 Lecture 6 的所有线索合在一起：tile 决定复用，mask 保证边界正确，accumulator 留在片上，activation fusion 避免额外 HBM traffic。Triton 让我们能用相对短的 Python 代码表达这些结构，但真正高性能仍依赖对硬件细节的持续 profile。

11 从硬件事实到工程判断

11.1 什么时候该写 kernel

写 kernel 的成本不只是代码量，还包括 correctness 测试、shape 覆盖、dtype 覆盖、autograd/backward、硬件兼容、未来 PyTorch/Triton 版本变化。一个保守但实用的判断顺序是：

1. 先用 PyTorch builtin 或成熟库实现。
2. 用 benchmark 确认它是否真的在目标 workload 中慢。
3. 用 profiler 确认慢在 kernel 数、HBM traffic、fallback path 还是 shape underutilization。

4. 尝试 `torch.compile`、调整 shape/layout、使用现成 fused operator。
5. 只有当瓶颈稳定、收益足够大、现成工具无法解决时，再写 Triton/CUDA。

kernel 优化的机会成本

一个 2% 的 kernel 加速，如果只影响总训练 step 的 5%，全局收益只有约 0.1%。相反，一个让 dominant kernel 少一次 HBM 往返的 fusion，可能直接改变 end-to-end throughput。优化前先看 profile 占比。

11.2 性能 checklist

问题	诊断信号	常见动作
Kernel 数太多	profiler 中大量 pointwise kernels	fusion、 <code>torch.compile</code> 、Triton fused kernel。
HBM traffic 过高	memory-bound，带宽接近上限	tiling、recomputation、减少中间写回。
小 shape latency 高	GPU utilization 低，launch 占比高	batch/sequence 合并、persistent kernel、减少 launch。
Shape 不友好	tile padding 多，kernel path 改变	调整 hidden size、padding 到硬件友好倍数。
Occupancy 低	register pressure 或 shared memory 过大	调小 tile、减少 live variables、评估是否真的需要提高 occupancy。
Bank conflict	shared memory stall 高	swizzling、padding、改变 shared layout。
Coalescing 差	global load efficiency 低	改 stride/layout，让 warp 访问连续地址。

11.3 Shape 本身是系统设计参数

LLM 架构设计常把 hidden size、head dim、MLP expansion、vocab padding 当成建模问题，但在 GPU 上它们也是系统参数。例如 head dimension 是否为 Tensor Core 友好的倍数，sequence length 是否导致 attention tile 尾部浪费，expert capacity 是否造成 MoE dispatch 不规则，vocab size 是否影响 logits/softmax kernel 效率。Lecture 6 的意义之一，就是提醒我们模型配置会改变 kernel path。

从算法到硬件的反向约束

不是所有数学上优雅的 shape 都是硬件友好的。大模型训练中，经常为了 Tensor Core、通信 chunk、KV cache alignment、FlashAttention tile 而选择或 padding 某些维度。这不是低级细节，而是可观的训练成本。

11.4 正确性测试不能只看一次 allclose

官方讲义中的 `check_equal_1d` 和 `check_equal_2d` 用 `torch.allclose` 对比 PyTorch reference。这是必要第一步，但高质量 kernel 还要覆盖：

- 非整除 shape，例如 $N = 1025$ 而 block size 为 1024；
- 极端数值，例如 softmax 中的大正数、大负数和 `-inf` mask；
- 不同 dtype，例如 fp32、bf16、fp16，以及 accumulation dtype；
- contiguous 与 non-contiguous strides；
- backward correctness，如果 kernel 进入训练路径。

性能优化不能破坏数值语义

Softmax、layernorm、attention 这类 reduction 算子尤其容易因为 dtype、mask 或稳定性处理出错。一个 kernel benchmark 很快但在极端 logits 下溢出，不能算优化成功。

11.5 本章小结

Lecture 6 给出的工程判断很克制：先用现成工具，测清楚，再有针对性地写 kernel。Triton 降低了手写 kernel 门槛，但没有取消 benchmark、profile、correctness 和维护成本。

12 拓展阅读

- [Stanford CS336 Spring 2026 course page](#)
- [Triton fused softmax tutorial](#)
- [PyTorch profiler recipe](#)
- [PyTorch benchmark recipe](#)
- [NVIDIA CUDA programming and PTX documentation](#)
- [CUTLASS documentation](#)

13 总结与延伸

这讲把 GPU 性能课从“硬件概念”推进到“可执行实验”。Benchmark 让我们知道速度，profile 让我们知道时间花在哪里，Triton 让我们在 block/tile 层表达 kernel，GeLU、softmax、row sum、matmul+ReLU 则覆盖了 pointwise、reduction、跨 tile reduction 和 tiled matrix multiply 四类核心模式。

最终 takeaways

1. GPU 优化必须从可信 benchmark 开始，异步执行和 warmup 会让朴素计时失真。
2. Profiler 是连接 PyTorch 表达式和 CUDA kernel 的证据层。
3. Fusion 的主要收益来自减少 HBM 中间读写，而不是改变数学计算。
4. Triton 的核心心智模型是 block/tile: load 到片上、融合计算、store 回 HBM。
5. Softmax 和 matmul 的高性能都依赖 reduction/tiling 状态设计。
6. 写 kernel 前先确认瓶颈足够重要、现成库/编译器无法解决、正确性测试覆盖充分。

和下一讲的连接

Lecture 6 关注单 GPU 内 kernel 如何跑快。下一讲进入多 GPU: 当一个 GPU 内的 HBM、SM 和 kernel 已经被认真优化后，新的瓶颈会变成跨 GPU 通信、并行策略、通信与计算重叠，以及训练系统的整体调度。